

Adaptive Model Routing & Eval-Gated Delivery for Claude Code

Oct 4, 2026 · @Mikkel

Summary

An Opus 5.5 orchestrator plans every change, scores each task against a fixed rubric, and dispatches it to the cheapest model tier likely to succeed. A separate eval agent verifies the work against acceptance criteria written at plan time, and a hook blocks every commit that lacks a passing eval for the exact diff being committed.

The gap this closes: Claude Code routes by configuration, not by task difficulty. Here, difficulty is judged once at plan time (the rubric) and again at run time (escalation triggers), and the second judgment is backed by hooks rather than left to the agent's memory.

Goals

- **Cost discipline:** default to the cheapest tier that gets the job done; Opus only where it pays.
- **Quality gate:** nothing is committed without an independent, reproducible eval.
- **Self-correcting routing:** wrong tier choices are caught and escalated, and the rubric is tuned from logged outcomes.
- **Mechanical enforcement:** the rules that matter most are hooks and permissions, not prose instructions.

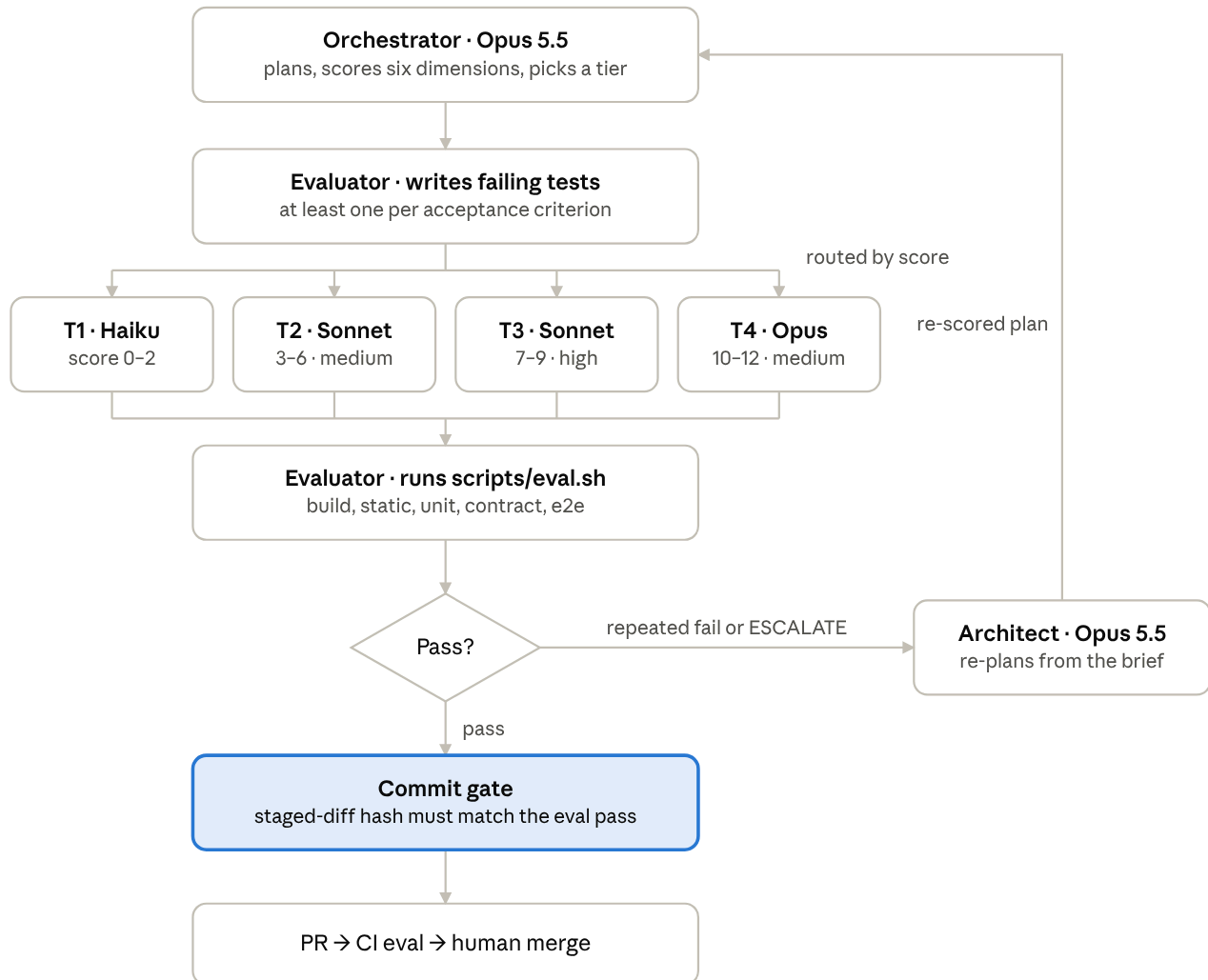
Non-goals

- Switching the main session's model mid-turn. The orchestrator stays on Opus; routing happens per subagent.
- Replacing CI. The eval agent runs the suite locally and keeps it in CI; CI stays the final authority on merge.
- Fully autonomous production deploys. Eval targets are local runs and preview environments only.

Architecture

One Opus orchestrator, four implementer tiers, an Opus architect for re-planning and an independent evaluator, wired together by hooks and a single eval script. The orchestrator never writes production code, and nothing it dispatches can commit without a fresh eval pass.

Every task is routed by score; only an eval pass reaches a commit



routing and eval flow · 4 tiers, 1 escalation loop, 1 gate

A task moves down the left column on the happy path. Repeated failures or an implementer's ESCALATE report send it right to the architect, whose re-scored plan goes back to the orchestrator for a fresh dispatch.

Repository layout

```
.claude/
  agents/
    orchestrator.md  impl-t1.md  impl-t2.md  impl-t3.md  impl-t4.md
    architect.md     evaluator.md
  skills/
    complexity-rubric/SKILL.md
    escalation-protocol/SKILL.md
  hooks/
    commit-gate.sh  marker-guard.sh  protect-tests.sh  tests-only.sh
    failure-counter.sh  require-report.sh  log-subagent.sh
  state/
    # gitignored: eval-pass.json, counters/, routing-
log.jsonl
  settings.json      # permissions + session-wide hooks
  scripts/
    eval.sh  test.sh  bootstrap.sh
  lefthook.yml        # pre-commit + commit-msg
  PLAN.md             # living plan: tasks, ACs, scores
  .github/workflows/eval.yml
```

Complexity rubric

Every task is scored on six dimensions, each 0, 1 or 2, giving a total of 0 to 12 that maps to one of four tiers. Six coarse judgments are more reliable than one fine-grained number, and each score carries a one-line justification so a wrong call can be traced.

Dimensions

Dimension	0	1	2
Ambiguity	Spec is complete; one obvious solution	Some decisions left open	Requirements unclear or conflicting
Blast radius	One file or module	Several modules in one service	Crosses a service boundary, public API or shared contract
Coupling	Isolated code	Touches shared utilities or config	Touches auth, payments, data model or messaging

Dimension	0	1	2
Novelty	Pattern exists in the repo	Pattern exists elsewhere, adapt it	New pattern, library or integration
Reversibility	Pure code change, revert is trivial	Needs coordinated revert (config, flags)	Data migration, schema change or external side effect
Verification difficulty	Unit tests cover it	Needs integration or e2e tests	Needs judgment: UX, performance, concurrency, security

Tiers

A tier is a model plus an effort level. Effort is a second, cheaper dial: Sonnet 5.5 at high effort often avoids a jump to Opus.

Tier	Score	Agent	Model	Effort	Typical work
T1	0-2	impl-t1	Haiku 4.5	n/a	Renames, boilerplate, docs, config edits, test fixtures
T2	3-6	impl-t2	Sonnet 5.5	medium	Most features and fixes inside one service
T3	7-9	impl-t3	Sonnet 5.5	high	Multi-module changes, new integrations with known patterns
T4	10-12	impl-t4	Opus 5.5	medium	Cross-service work, novel design under uncertainty

Override rules

These apply after scoring and only ever raise the tier:

- Any **2 on Reversibility** → at least T3, and the plan must include a rollback step.
- Any **2 on Coupling** where security is involved (auth, secrets, permissions) → T4 and a human review flag on the PR.
- **Two or more 2s** anywhere → at least T3.
- A task the orchestrator cannot score confidently is split, not rounded up.

Examples

Task	Scores (A·B·C·N·R·V)	Total	Tier
Rename a DTO field used in one service	0·1·0·0·0·0	1	T1
Add an endpoint following an existing controller pattern	0·1·1·0·0·1	3	T2
Add retry and dead-letter handling to a message consumer	1·1·2·1·1·2	8	T3
Introduce a new event contract consumed by three services	2·2·2·2·1·2	11	T4

The thresholds above are starting values. The Telemetry section describes how they get tuned from real outcomes.

Agent definitions

Seven agent files live in `.claude/agents/` : one orchestrator, four implementer tiers, one architect and one evaluator. Each tier is its own file rather than one implementer with a per-call model override, because a file pins model **and** effort together, and its name shows up as `agent_type` in every hook and log line.

Agent	Model	Effort	Role	Writes code?
<code>orchestrator</code>	Opus 5.5	high	Plans, scores, dispatches, decides escalations	No (only <code>PLAN.md</code> and the routing log)
<code>impl-t1</code>	Haiku 4.5	n/a	Mechanical changes	Yes, outside test files
<code>impl-t2</code>	Sonnet 5.5	medium	Standard implementation	Yes, outside test files
<code>impl-t3</code>	Sonnet 5.5	high	Harder implementation	Yes, outside test files
<code>impl-t4</code>	Opus 5.5	medium	Hardest implementation	Yes, outside test files

Agent	Model	Effort	Role	Writes code?
architect	Opus 5.5	high	Re-plans after an escalation	No (only PLAN.md)
evaluator	Opus 5.5	medium	Writes tests from acceptance criteria, runs the harness, judges results	Test files only

Orchestrator

The orchestrator runs as the main session through `claude --agent orchestrator` (or `"agent": "orchestrator"` in `.claude/settings.json`). Running it as the main-thread agent unlocks the `Agent(type, ...)` allowlist, so it can only spawn the six agents above.

```

---
name: orchestrator
description: Plans work, scores task complexity, routes tasks to the cheapest
capable tier, gates commits on eval.
model: opus
effort: high
tools: Agent(impl-t1, impl-t2, impl-t3, impl-t4, architect, evaluator), Read,
Grep, Glob, Write, Edit, Bash
skills:
  - complexity-rubric
  - escalation-protocol
---
```

You are the orchestrator. You do not write production code.

For every request:

1. Write or update PLAN.md: tasks, acceptance criteria per task, dependencies.
2. Score each task with the complexity-rubric skill. Record scores and the one-line justification per dimension in PLAN.md.
3. Ask the evaluator to write failing tests from the acceptance criteria.
4. Dispatch each task to the impl-tN agent its tier names. Pass the task, its acceptance criteria, and the relevant PLAN.md section.
5. When an implementer reports DONE, ask the evaluator to run the harness.
6. On ESCALATE or repeated eval failure, follow the escalation-protocol skill.
7. Commit only after the evaluator reports PASS for the current diff.

8. Append one line per task outcome to `.claude/state/routing-log.jsonl`.

Prefer the cheapest tier the rubric allows. Never raise a tier "to be safe"; split the task instead.

Implementer tiers

The four files differ only in `name`, `model`, `effort` and `description`. They share one system prompt and one set of frontmatter hooks.

```
---
name: impl-t2
description: Standard implementation tier. Use for tasks scored 3 to 6.
model: sonnet
effort: medium
tools: Read, Grep, Glob, Edit, Write, Bash
maxTurns: 60
hooks:
  PreToolUse:
    - matcher: "Edit|Write"
      hooks:
        - type: command
          command: "${CLAUDE_PROJECT_DIR}/.claude/hooks/protect-tests.sh"
  PostToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "${CLAUDE_PROJECT_DIR}/.claude/hooks/failure-counter.sh"
  Stop:
    - hooks:
        - type: command
          command: "${CLAUDE_PROJECT_DIR}/.claude/hooks/require-report.sh"
---
```

Implement exactly the task you are given. Do not change test files.

Finish with one of two reports, and nothing else:

- DONE: what changed, files touched, how you checked it locally.
- ESCALATE: the escalation brief from the escalation-protocol skill.

Escalate immediately, without further attempts, when a hook tells you an escalation threshold was hit or when the plan's assumptions turn out wrong.

- `impl-t1: model: haiku, no effort line, maxTurns: 30.`
- `impl-t3: model: sonnet, effort: high, maxTurns: 80.`
- `impl-t4: model: opus, effort: medium, maxTurns: 80.`

Architect

Called only during escalation. It receives the escalation brief and returns an updated `PLAN.md` section with re-scored tasks.

```
---
name: architect
description: Re-plans a task after an implementer escalates. Returns a
revised plan, never code.
model: opus
effort: high
tools: Read, Grep, Glob, Write, Bash
---
```

You receive an escalation brief. Read the code it points at, find the faulty assumption, and rewrite the affected `PLAN.md` section: smaller tasks, explicit interfaces, revised acceptance criteria, fresh rubric scores. Do not edit source files. End with a short summary of what changed in the plan and why.

Evaluator

The evaluator is deliberately separate from the implementers: it writes tests from the plan's acceptance criteria before or alongside implementation, so the tests describe the requirement rather than the code that was written.

```
---
name: evaluator
description: Writes and extends tests from acceptance criteria, runs the eval
harness, reports PASS or FAIL with evidence.
model: opus
effort: medium
tools: Read, Grep, Glob, Edit, Write, Bash
mcpServers:
  - playwright:
      type: stdio
      command: npx
      args: ["-y", "@playwright/mcp@latest"]
```

```
hooks:
  PreToolUse:
    - matcher: "Edit|Write"
      hooks:
        - type: command
          command: "${CLAUDE_PROJECT_DIR}/.claude/hooks/tests-only.sh"
---
```

Two modes, named in your task:

WRITE-TESTS: derive tests from the acceptance criteria in PLAN.md. Every criterion gets at least one test. Tests must fail before implementation.

EVALUATE: run scripts/eval.sh and read its report. Return PASS, or FAIL with the failing checks, the likely cause, and which acceptance criterion is unmet.

Never edit source files to make a test pass.

Escalation

The plan-time score is a prediction; escalation is the correction. Triggers are counted by hooks, the implementer stops and writes a brief, the architect re-plans on Opus, and the orchestrator re-dispatches at whatever tier the new scores call for. A task escalates at most twice; the third time it stops and asks a human.

Triggers

Trigger	Detected by	Threshold (starting value)
Same test command fails after attempted fixes	failure-counter.sh (PostToolUse on Bash)	3 consecutive failures
Same file edited repeatedly without a passing run	failure-counter.sh (PostToolUse on Edit/Write)	6 edits to one file
Change needs files outside the task's declared scope	Implementer self-report	Any
Plan assumption proven wrong (missing API, wrong data shape)	Implementer self-report	Any

Trigger	Detected by	Threshold (starting value)
Task needs a trade-off, new contract or eval design	Implementer self-report	Any
Evaluator FAIL on the same acceptance criterion	Orchestrator	2 eval rounds
Implementer hits <code>maxTurns</code>	Claude Code (output marked partial)	Per tier file

How the counting hook works

Test runs go through one wrapper, `scripts/test.sh`, which writes its exit code to `.claude/state/last-test-exit`. The hook reads that file instead of parsing tool output, so it doesn't depend on the shape of Bash results.

```
#!/bin/bash
# .claude/hooks/failure-counter.sh (PostToolUse, implementer frontmatter)
input=$(cat)
agent=$(jq -r '.agent_id // "main"' <<<"$input")
tool=$(jq -r '.tool_name' <<<"$input")
state="${CLAUDE_PROJECT_DIR}/.claude/state/counters/${agent}.json"
mkdir -p "$(dirname "$state")"; [ -f "$state" ] || echo '{"fails":0,"edits":{}}' > "$state"

if [ "$tool" = "Bash" ]; then
  cmd=$(jq -r '.tool_input.command' <<<"$input")
  [[ "$cmd" == *scripts/test.sh* ]] || exit 0
  code=$(cat "${CLAUDE_PROJECT_DIR}/.claude/state/last-test-exit" 2>/dev/null || echo 0)
  if [ "$code" != "0" ]; then
    fails=$(jq '.fails + 1' "$state"); jq ".fails = $fails" "$state" > "$state.tmp" && mv "$state.tmp" "$state"
    if [ "$fails" -ge 3 ]; then
      echo "Escalation threshold reached: tests failed $fails times in a row. Stop and return an ESCALATE report." >&2
      exit 2 # PostToolUse exit 2 shows stderr to the agent
    fi
  else
    jq '.fails = 0 | .edits = {}' "$state" > "$state.tmp" && mv "$state.tmp"
```

```
"$state"
  fi
fi
# Edit/Write branch: same pattern, counting .edits[file_path] against 6
exit 0
```

A `require-report.sh` hook on the implementer's `Stop` (run as `SubagentStop`) checks that the final message starts with `DONE:` or `ESCALATE:` and blocks the stop otherwise, so an implementer can't drift off without a usable result.

Escalation brief

The architect starts with no conversation history, so the brief is the entire handoff. The `escalation-protocol` skill holds this template:

```
ESCALATE: <task id> – <one-line reason>

Trigger: <which trigger, with the count or evidence>
Task and tier: <task text>, scored <A·B·C·N·R·V = total>, ran on impl-tN
What was tried: <2-5 bullets, each with the outcome>
Current failure: <exact error or failing test names, trimmed>
Suspected wrong assumption: <the plan statement that no longer holds>
Files touched: <paths>
Question for the architect: <the one decision needed>
```

Re-dispatch and de-escalation

1. The orchestrator passes the brief to `architect`.
2. The architect rewrites the affected `PLAN.md` section with smaller tasks and fresh scores.
3. The orchestrator dispatches each new task at its **new** tier. Re-planning usually lowers the score of each piece, so tasks often drop back to T2 or T1.
4. Counters for the task reset; the escalation count does not.
5. On a second escalation of the same original task, the orchestrator stops, summarises both briefs, and asks the human.

Alternative worth testing: the advisor tool

Claude Code has an [advisor tool](#) that lets the running model consult a second, stronger model mid-task instead of switching at a plan boundary. It could replace a full architect round-trip for small questions. Phase 5 of the implementation plan compares the two.

Eval harness

The harness is one deterministic script, `scripts/eval.sh`, that runs identically on a laptop and in CI. The evaluator agent decides what to test and interprets failures; the script does the running and is the only thing allowed to write the pass marker.

Evaluator model per tier

The judgment part is where Opus pays off, but routine tasks don't need it. The orchestrator passes the Agent tool's per-call `model` parameter, which overrides the evaluator's frontmatter:

Task tier	Evaluator model
T1, T2	Sonnet 5.5
T3, T4, any escalated task	Opus 5.5 (frontmatter default)

Test derivation

1. Each task in `PLAN.md` carries numbered acceptance criteria (`AC-1` , `AC-2` , ...).
2. In WRITE-TESTS mode the evaluator writes at least one test per criterion and names it after the ID (`test_AC_2_rejects_expired_token`).
3. The tests are run once and must fail. A test that passes before implementation is reported as not testing anything.
4. Implementers cannot edit test files (`protect-tests.sh`); the evaluator can only edit test files (`tests-only.sh`).
5. Tests are committed together with the code, so the suite grows with every change.

Check stages

`scripts/eval.sh` runs these in order and stops at the first failure. Each stage is skipped cleanly when a project doesn't have it.

Stage	What it runs	Runs in CI
Build	Project build (<code>dotnet build</code> , <code>npm run build</code> , ...)	Yes
Static	Lint, format check, type check	Yes
Unit	Unit tests, including the new AC tests	Yes
Contract	Consumer-driven contract tests for any changed message or API contract	Yes
Integration	Service plus real dependencies in containers	Yes
E2E	Playwright against a local run or a Vercel preview URL	Yes
Exploratory	Agent-driven browser check of the changed flow	No

On success the script writes `.claude/state/eval-pass.json` with the staged-diff hash, the `HEAD` commit, a timestamp and per-stage results. The Commit gate section explains how that marker is used.

Browser testing: Playwright first, Chrome for exploration

- **Playwright** (scripted specs, plus the Playwright MCP server in the evaluator’s frontmatter for authoring) is the default for anything that must be repeatable. Specs live in the repo and run in CI.
- **Claude in Chrome** is for the Exploratory stage: clicking through a changed flow the way a user would, catching layout or copy problems no spec covers. Findings are turned into Playwright specs before the task closes, so nothing important depends on a run that CI can’t repeat.

Support CLIs

CLI	Used for	Allowed	Denied
<code>gh</code>	PRs, CI status, run logs	<code>gh pr create</code> , <code>gh pr view</code> , <code>gh run list</code> , <code>gh run view</code> , <code>gh run watch</code>	<code>gh pr merge</code> , <code>gh release</code> , <code>gh repo delete</code> , <code>gh secret</code>

CLI	Used for	Allowed	Denied
<code>az</code>	Read-only checks of Azure resources, app settings, logs	<code>az ... show</code> , <code>az ... list</code> , <code>az monitor ...query</code>	<code>az ... create</code> , <code>az ... delete</code> , <code>az ... update</code> , <code>az role ...</code> , <code>az keyvault secret ...</code>
<code>vercel</code>	Preview deployments as e2e targets	<code>vercel deploy (preview)</code> , <code>vercel inspect</code> , <code>vercel logs</code>	<code>vercel --prod</code> , <code>vercel promote</code> , <code>vercel env rm</code> , <code>vercel remove</code>

The `az` CLI should authenticate as a dedicated service principal with the Reader role on the relevant resource groups, so the permission rules are a second line of defence rather than the only one.

CI

The same script runs on every pull request, which makes CI the final authority and keeps local and CI results comparable.

```
# .github/workflows/eval.yml
name: eval
on: [pull_request]
jobs:
  eval:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install dependencies
        run: ./scripts/bootstrap.sh
      - name: Run eval harness
        run: ./scripts/eval.sh --ci
      - name: Upload report
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: eval-report
          path: .eval/report/
```

In `--ci` mode the script skips the Exploratory stage and doesn't write the local pass marker. Branch protection on `main` requires the `eval` check.

Commit gate and hooks

A commit goes through only when `.claude/state/eval-pass.json` holds the hash of exactly the staged diff being committed and is less than 30 minutes old. Two independent layers check this, so a commit that slips past one is caught by the other.

Sequence

1. Stage the change: `git add -A`.
2. Run `scripts/eval.sh`. On success it writes the marker with `sha256(git diff --cached --binary)`.
3. Commit. Both gates recompute the hash and compare.
4. Any edit after step 2 changes the hash, so the commit is refused until eval runs again.

Layer 1: Claude Code PreToolUse hook

Registered in `.claude/settings.json`, so it covers the main session and every subagent:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "if": "Bash(git commit *)",
            "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/commit-gate.sh",
            "args": []
          }
        ]
      }
    ]
  }
}
```

```
#!/bin/bash
# .claude/hooks/commit-gate.sh
marker="${CLAUDE_PROJECT_DIR}/.claude/state/eval-pass.json"
deny() {
  jq -n --arg r "$1" '{hookSpecificOutput:
{hookEventName:"PreToolUse",permissionDecision:"deny",permissionDecisionReason:$r}}}'
  exit 0
}
[ -f "$marker" ] || deny "No eval pass recorded. Run scripts/eval.sh on the
staged change first."
staged=$(git -C "$CLAUDE_PROJECT_DIR" diff --cached --binary | sha256sum |
cut -d' ' -f1)
recorded=$(jq -r '.diff_sha256' "$marker")
age=$(( $(date +%s) - $(jq -r '.timestamp' "$marker") ))
[ "$staged" = "$recorded" ] || deny "Staged changes differ from what was
evaluated. Re-run scripts/eval.sh."
[ "$age" -le 1800 ] || deny "Eval pass is older than 30 minutes. Re-run
scripts/eval.sh."
exit 0
```

The `if` filter is best-effort: Claude Code runs the hook whenever it can't tell what a Bash command does, which is safe here because the script only ever denies. The docs are explicit that hard enforcement belongs in permission rules and real git hooks, which is why Layer 2 exists.

Layer 2: native git hooks

Installed with `lefthook` (or a plain `core.hooksPath`) so they apply to every commit, however it is made:

- **pre-commit** runs the same hash and age check as `commit-gate.sh` and exits non-zero on mismatch.
- **commit-msg** strips AI attribution from every commit message: `Co-Authored-By: Claude ...` trailers, `Generated with Claude Code` lines and `session-URL` trailers. It rewrites the message file in place instead of rejecting the commit.

Humans can still bypass Layer 2 with `git commit --no-verify` for emergencies. Claude cannot: `--no-verify` is in the permission deny list. Also check the current Claude Code settings reference for its attribution setting and turn attribution off there, so the `commit-msg` hook is a backstop rather than the main mechanism.

Protecting the marker

- `Edit` and `Write` on `.claude/state/**` are denied in permissions, so only `scripts/eval.sh` writes there.
- A `PreToolUse` hook refuses Bash commands that mention `eval-pass.json` other than through `scripts/eval.sh`.
- This is a guardrail against an agent taking a shortcut, not a security boundary. CI re-running the full suite on the PR is the boundary.

Hook inventory

Hook script	Event	Where registered	Purpose
<code>commit-gate.sh</code>	<code>PreToolUse</code> (Bash, <code>git commit *</code>)	<code>.claude/settings.json</code>	Deny commits without a matching eval pass
<code>marker-guard.sh</code>	<code>PreToolUse</code> (Bash)	<code>.claude/settings.json</code>	Refuse direct writes to the pass marker
<code>protect-tests.sh</code>	<code>PreToolUse</code> (Edit, Write)	Implementer frontmatter	Implementers can't edit test files
<code>tests-only.sh</code>	<code>PreToolUse</code> (Edit, Write)	Evaluator frontmatter	Evaluator can only edit test files
<code>failure-counter.sh</code>	<code>PostToolUse</code> (Bash, Edit, Write)	Implementer frontmatter	Count failures and repeated edits; signal escalation
<code>require-report.sh</code>	<code>Stop</code> → <code>SubagentStop</code>	Implementer frontmatter	Require a DONE or ESCALATE report
<code>log-subagent.sh</code>	<code>SubagentStart</code> , <code>SubagentStop</code>	<code>.claude/settings.json</code>	Append start, stop and duration to the routing log
<code>pre-commit</code>	<code>git</code>	lefthook	Second hash check, outside Claude Code
<code>commit-msg</code>	<code>git</code>	lefthook	Strip AI attribution trailers

All policy hooks exit 2 or return a JSON deny to block. Exit 1 does not block in Claude Code; it is treated as a non-blocking error and the action proceeds.

Permissions and safety

Permission rules are the hard boundary; hooks and prompts are softer layers on top. The rules below go in `.claude/settings.json` and apply to the main session and every subagent.

```
{
  "permissions": {
    "allow": [
      "Bash(/scripts/eval.sh*)",
      "Bash(/scripts/test.sh*)",
      "Bash(git status*)", "Bash(git diff*)", "Bash(git add *)", "Bash(git
commit *)",
      "Bash(git push origin feat/*)", "Bash(git push origin fix/*)",
      "Bash(gh pr create*)", "Bash(gh pr view*)", "Bash(gh run list*)",
      "Bash(gh run view*)", "Bash(gh run watch*)",
      "Bash(az * show*)", "Bash(az * list*)", "Bash(az monitor * query*)",
      "Bash(vercel deploy)", "Bash(vercel inspect*)", "Bash(vercel logs*)"
    ],
    "deny": [
      "Edit(.claude/state/**)", "Write(.claude/state/**)",
      "Edit(.claude/settings.json)", "Edit(.claude/hooks/**)",
      "Edit(.github/workflows/**)",
      "Bash(git commit --no-verify*)", "Bash(git commit * --no-verify*)",
      "Bash(git commit -n*)",
      "Bash(git push --force*)", "Bash(git push * --force*)", "Bash(git push
* main*)",
      "Bash(git reset --hard*)",
      "Bash(gh pr merge*)", "Bash(gh release*)", "Bash(gh repo delete*)",
      "Bash(gh secret*)",
      "Bash(az * create*)", "Bash(az * delete*)", "Bash(az * update*)",
      "Bash(az role*)", "Bash(az keyvault secret*)",
      "Bash(vercel --prod*)", "Bash(vercel deploy --prod*)", "Bash(vercel
promote*)", "Bash(vercel remove*)", "Bash(vercel env rm*)"
    ]
  }
}
```

Principles

- **No production writes from any agent.** Merges to `main`, production deploys and cloud resource changes stay human actions.
- **Least-privilege credentials.** The `az` service principal is Reader-only; the Vercel token is scoped to preview deploys; the GitHub token can open PRs but not merge them. If a rule is missed, the credential still says no.
- **Agents can't edit their own guardrails.** Settings, hook scripts, CI workflows and the state directory are all write-denied.
- **Permission mode.** Run in Manual or `acceptEdits` mode, not `bypassPermissions`. Subagents inherit the main session's mode in `bypassPermissions`, `acceptEdits` and auto mode, so the main session's choice is the effective choice for the whole team.
- **CI is the final boundary.** Deny patterns are string matches and can be sidestepped by unusual command shapes; branch protection plus the CI `eval` check catches anything that gets through locally.

Telemetry and rubric calibration

Every task writes one line to `.claude/state/routing-log.jsonl`, and a weekly review compares predicted tier with actual outcome. After roughly 50 logged tasks the thresholds are tuned from data rather than intuition.

Routing log record

The orchestrator writes the outcome fields; `log-subagent.sh` fills in agent IDs and timings from `SubagentStart` and `SubagentStop`.

```
{
  "task_id": "PLAN-42.3",
  "date": "2026-11-12",
  "scores": {"ambiguity": 1, "blast": 1, "coupling": 2, "novelty": 1,
"reversibility": 1, "verification": 2},
  "total": 8,
  "overrides": [],
  "tier_planned": "T3",
  "agents": ["impl-t3", "evaluator"],
  "eval_rounds": 2,
  "escalations": 0,
  "final_tier": "T3",
  "outcome": "pass",
  "duration_s": 1140,
  "diff_lines": 212
}
```

outcome is one of pass_first_try , pass , escalated_then_pass , human . Cost per task comes from Claude Code’s OpenTelemetry export or session cost reports joined on time and agent ID.

Weekly review

Metric	What it tells you	Starting target
First-try pass rate per tier	Whether tasks land in a tier that can handle them	≥ 70%
Escalation rate per tier	Whether a tier’s upper threshold is too high	≤ 15%
Eval rounds per task	Whether acceptance criteria and tests are clear	≤ 2 average
Human interventions	Whether the system is stuck on a class of task	Trending down
Cost per task by tier	Whether routing actually saves money	T2 well below T4

Calibration rules

1. **Escalation rate above 25% in a tier** → lower that tier's upper threshold by 1 point, so borderline tasks move up.
2. **First-try pass above 90% in T3 or T4, with small diffs** → raise the lower threshold by 1, so borderline tasks move down to a cheaper tier.
3. **One dimension dominates escalations** (for example most escalated tasks scored low on Coupling but touched messaging) → fix the dimension's wording or add an override rule.
4. **Change one threshold at a time** and leave it for at least a week, so the effect can be measured.
5. Record each change and its reason in the `complexity-rubric` skill's changelog.

Implementation plan

Build the eval harness and commit gate first: they deliver value on their own and everything else depends on them. Routing comes second, escalation third, and calibration only once there is data. Each phase ends with exit criteria that can be ticked off; sizes are rough estimates for one engineer.

Phase 0 — Prerequisites (about half a day)

Pick one pilot repository, ideally a single service with decent existing tests.

- ☐ Claude Code at v2.1.284 or later on every machine (needed for Sonnet 5.5; Opus 5.5 needs v2.1.280)
- ☐ `jq`, `lefthook` and Playwright installed; `scripts/bootstrap.sh` installs them in CI
- ☐ Azure service principal with Reader role on the pilot's resource groups
- ☐ Vercel token scoped to preview deploys (if the pilot has a frontend)
- ☐ Branch protection on `main` requiring PR review and a status check

Phase 1 — Eval harness and commit gate (2–3 days)

Deliverables: `scripts/eval.sh`, `scripts/test.sh`, `.github/workflows/eval.yml`, `commit-gate.sh`, `marker-guard.sh`, `lefthook pre-commit` and `commit-msg`, permission rules from the Permissions section.

- ☐ `eval.sh` runs all applicable stages locally and in CI with identical results on the pilot repo

- ☐ A commit with no eval pass is denied, with the reason shown to Claude
- ☐ Editing a file after eval, then committing, is denied until eval re-runs
- ☐ A commit made outside Claude Code is also blocked by `pre-commit`
- ☐ Commit messages containing attribution trailers come out clean
- ☐ The CI `eval` check is required on `main`

Phase 2 — Agents and static routing (2 days)

Deliverables: the seven agent files, the `complexity-rubric` skill, the `PLAN.md` template, `protect-tests.sh` and `tests-only.sh`.

- ☐ `claude --agent orchestrator` can only spawn the six listed agents
- ☐ Five sample tasks are scored with justifications and dispatched to the matching tier; `/tasks` shows the expected model for each
- ☐ Evaluator writes failing AC tests before implementation; implementers are blocked from editing them
- ☐ Orchestrator passes `model: sonnet` to the evaluator for T1 and T2 tasks
- ☐ One full loop (plan → tests → implement → eval → commit) completes on a real task

Phase 3 — Escalation loop (2 days)

Deliverables: `failure-counter.sh`, `require-report.sh`, the `escalation-protocol` skill with the brief template, architect re-plan flow.

- ☐ Three forced test failures trigger the threshold message and an ESCALATE report
- ☐ An implementer that stops without DONE or ESCALATE is sent back to finish
- ☐ The architect returns a re-scored plan from a brief alone, with no other context
- ☐ Re-dispatched tasks run at their new tier; a second escalation stops and asks the human

Phase 4 — Telemetry and hardening (1 day)

Deliverables: `log-subagent.sh`, routing-log writes in the orchestrator, a small script that prints the weekly metrics table.

- ☐ Every completed task has a complete routing-log line
- ☐ The weekly report prints all five metrics from the Telemetry section
- ☐ A red-team pass tries to bypass each guardrail (marker write, `--no-verify`, test edits, production deploy); findings are fixed or recorded as accepted risks

Phase 5 — Pilot and calibration (2-4 weeks, part-time)

- ☐ At least 50 tasks logged on the pilot repo
- ☐ Thresholds adjusted at most once per week, each change recorded in the rubric changelog
- ☐ Advisor tool compared with the architect round-trip on 10 escalations: resolution rate, time and cost
- ☐ Go or no-go decision for wider rollout, written down with the metrics behind it

Phase 6 — Rollout

- ☐ Package the setup for other repositories. If it ships as a plugin, move the subagent frontmatter hooks into the plugin's `hooks/hooks.json`, because plugin subagents ignore `hooks`, `mcpServers` and `permissionMode` in their frontmatter
- ☐ Per-repo `eval.sh` stage configuration documented
- ☐ Short team guide: how to read a `PLAN.md`, how to override a tier, how to bypass the gate in an emergency

Risks and open questions

The biggest risk is that the orchestrator itself becomes the main cost, since it runs on Opus for the whole session. The rest are manageable with the mitigations listed.

Risks

Risk	Impact	Mitigation
Orchestrator on Opus dominates cost in long sessions	Savings from cheap tiers are eaten up	Keep the orchestrator lean (no code reading beyond scoping); try <code>effort: medium</code> in Phase 5; compare cost per task with telemetry
Inconsistent scoring between sessions	Same task lands in different tiers	Justification per dimension, worked examples in the skill, calibration from the log

Risk	Impact	Mitigation
Tests written to pass rather than to verify	Gate passes broken code	Evaluator writes tests from ACs before implementation; tests must fail first; implementers can't edit tests
Subagent reports flood the orchestrator's context	Orchestrator quality drops over a long session	Strict DONE and ESCALATE formats; detail stays in subagent transcripts
Handoff loses context	Re-plans miss the real problem	Mandatory brief template; <code>require-report.sh</code> blocks reports without one
Latency of the full loop on small changes	Developers bypass the system	T1 path with a reduced eval (build, static, unit); human override documented
Guardrails sidestepped by unusual command shapes	A commit or deploy slips through locally	Native git hooks, scoped credentials, CI as final boundary

Open questions to settle during implementation

- `if: "Bash(git commit *)"` won't match `git -C <path> commit`. Layer 2 catches it, but decide whether to add a broader matcher or deny `git -C ... commit` outright.
- Confirm in Phase 2 that the orchestrator reliably passes the per-call `model` parameter for the evaluator; if not, split the evaluator into two files like the implementer tiers.
- Check the name of Claude Code's attribution setting in the current settings reference and set it, so the `commit-msg` hook is only a backstop.
- Decide whether T1 should run on Haiku 4.5 or on Sonnet 5.5 at low effort. Haiku is cheaper; Sonnet at low effort may need fewer eval rounds. Measure both in the pilot.
- Decide whether subagents run in the background (the interactive default) or foreground. The orchestrator needs each result before its next step, so it must wait for completion notifications rather than polling.
- For multi-service changes, decide whether contract tests run against a broker or against checked-in contract files.

Verification checklist before first real use

- ☐ Every hook script is executable and its path resolves; a mistyped path silently disables a gate
- ☐ Each policy hook blocks with exit 2 or a JSON deny, never exit 1
- ☐ The project folder is trusted, so subagent frontmatter hooks actually run
- ☐ `/hooks` lists every hook from the inventory table with the expected source
- ☐ `/tasks` shows the expected model on each tier's row

Sources

- [Hooks reference](#): events, exit codes, JSON decisions, hooks in skill and subagent frontmatter
- [Create custom subagents](#): frontmatter fields, model resolution order, `Agent(type)` allowlist, nesting limits
- [Model configuration](#): aliases, `opusplan`, effort levels per model, version requirements
- [Advisor tool](#): mid-task consultation of a second model